

Algorithm

Design a BankAccount Class Ensuring Sufficient Balance for Withdrawal

Design a BankAccount Class

Use Case

Design a Bankaccount class that represents a real-world bank account and allows safe financial transactions such as deposit and withdrawal.

Objective

To create a bank account system using Object-Oriented Programming concepts and ensure that **withdrawal is not allowed when the balance is insufficient**.

Class Description

The BankAccount class acts as a blueprint for creating bank account objects. It stores account details and provides methods to perform banking operations.

Properties (Data Members)

1. **account_number**
Stores the unique account number of the customer.
2. **account_holder_name**
Stores the name of the account holder.
3. **balance**
Stores the current available balance in the account.

Behaviors (Member Functions)

1. deposit(amount)

- Adds the specified amount to the account balance.
- Accepts only positive values.
- Prevents invalid deposit transactions.

2. withdraw(amount)

- Removes money from the account.
- Check whether the withdrawal amount is positive.
- Verifies if the account has **sufficient balance** before withdrawing.
- If the balance is insufficient, the withdrawal is rejected.
- This ensures safe and secure transactions.

3. display_balance()

- Displays account number, account holder name, and current balance.
- Helps the user track account details easily.

Working Process

1. A Bank account object is created using account details.
2. The user interacts with the system through a menu-driven interface.
3. Based on the user's choice, deposit or withdrawal operations are performed.
4. Before withdrawing, the system checks for sufficient balance.
5. If conditions are satisfied, the transaction is completed successfully.

Conclusion

The Bank account class provides a secure and structured way to manage bank accounts.

By restricting withdrawals when the balance is insufficient, it ensures financial safety and demonstrates effective use of Object-Oriented Programming concepts.

DFD

Level 1 Data Flow Diagram: BankAccount Class

Entity/Component	Description
External Entity	Account Holder / User (Initiates actions)
Processes	1.0 Deposit, 2.0 Withdraw, 3.0 Display Balance
Data Store	Accounts Database (Stores account_number, account_holder_name, balance)

Data Flows by Process

1. 1.0 Deposit(amount)

- **Input:** User provides deposit_amount.
- **Logic:** Process 1.0 receives the amount and the current balance from the Data Store.
- **Update:** New balance (balance + amount) is sent back to the Data Store.
- **Output:** "Deposit Successful" message to the User.

2. 2.0 Withdraw(amount)

- **Input:** User provides withdrawal_amount.
- **Validation:** Process 2.0 retrieves the current balance from the Data Store.
- **Decision Logic:**
 - **If** amount <= balance: New balance (balance - amount) is saved to the Data Store.
 - **If** amount > balance: An "Insufficient Funds" error message is sent to the User; no update is made to the Data Store.

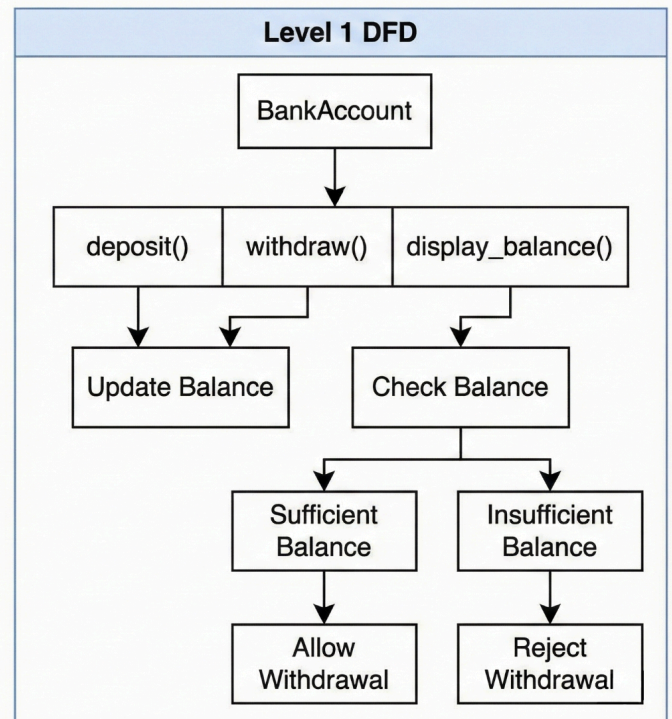
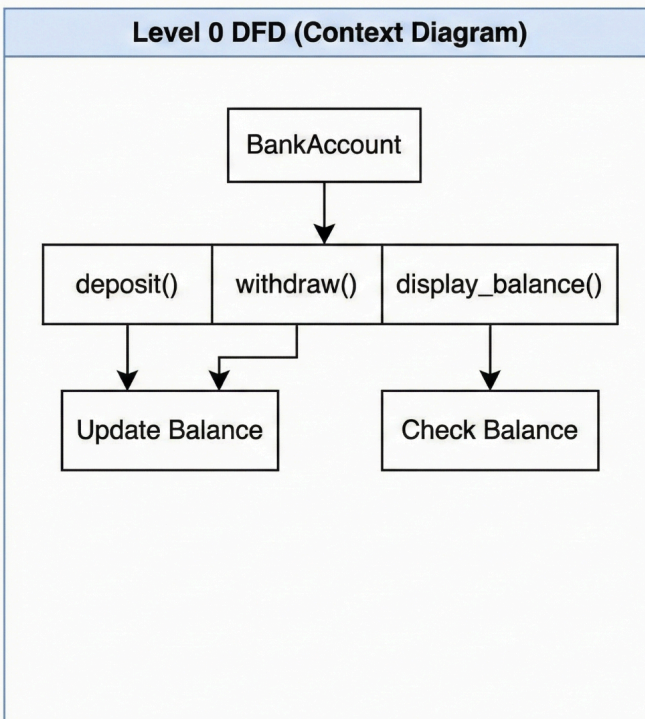
- **Output:** Success or Failure confirmation to the User.

3. 3.0 Display Balance

- **Request:** User requests "Display Balance".
- **Retrieval:** Process 3.0 fetches `account_number`, `name`, and `balance` from the Data Store.
- **Output:** Account summary (all properties) is displayed to the User.

Key Components for Design

- **Processes (Circles/Rounded Rectangles):** Represent the `deposit()`, `withdraw()`, and `display_balance()` behaviors.
- **Data Store (Open-ended Rectangles):** Represents the `BankAccount` object's memory or a database where properties are held.
- **Data Flows (Arrows):** Show the specific pieces of information (like `amount` or `status`) moving between entities and processes.



Flowchart

FLOWCHART for Bank Account Program

